WriteUp

0X00

给了一个 rise 和 core 文件，用 file 分析知道一个是可执行文件，一个是核心转储文件，然后用 gdb -q ./rise ./core 进行分析，知道了它的启动命令：

```
Core was generated by `./rise flag'.
```

然后让我们看看它的内存布局吧。

```
Start              End Perm     Size  Offset File (set vmmap-prefer-relpaths on)
0x564e4b2da000     0x564e4b2db000 r--p     1000       0 /mnt/rise
0x564e4b2db000     0x564e4b2dc000 r-xp     1000    1000 /mnt/rise
0x564e4b2dc000     0x564e4b2dd000 r--p     1000    2000 /mnt/rise
0x564e4b2dd000     0x564e4b2de000 r--p     1000    2000 /mnt/rise
0x564e4b2de000     0x564e4b2df000 ---p     1000    3000 /mnt/rise
0x564e4bea3000     0x564e4bec4000 ---p    21000       0 [load6]
0x7f7067d52000     0x7f7067d74000 r--p    22000       0 /lib/x86_64-linux-gnu/libc-2.31.so
0x7f7067d74000     0x7f7067ecd000 r-xp   159000   22000 /lib/x86_64-linux-gnu/libc-2.31.so
0x7f7067ecd000     0x7f7067f1c000 r--p     4f000  17b000 /lib/x86_64-linux-gnu/libc-2.31.so
0x7f7067f1c000     0x7f7067f20000 r--p     4000  1c9000 /lib/x86_64-linux-gnu/libc-2.31.so
0x7f7067f20000     0x7f7067f22000 ---p     2000  1cd000 /lib/x86_64-linux-gnu/libc-2.31.so
0x7f7067f22000     0x7f7067f28000 ---p     6000       0 [load12]
0x7f7067f2b000     0x7f7067f2c000 r--p     1000       0 /lib/x86_64-linux-gnu/ld-2.31.so
0x7f7067f2c000     0x7f7067f4c000 r-xp    20000    1000 /lib/x86_64-linux-gnu/ld-2.31.so
0x7f7067f4c000     0x7f7067f54000 r--p     8000   21000 /lib/x86_64-linux-gnu/ld-2.31.so
0x7f7067f54000     0x7f7067f55000 ---p     1000       0 /mnt/flag
0x7f7067f55000     0x7f7067f56000 r--p     1000   29000 /lib/x86_64-linux-gnu/ld-2.31.so
0x7f7067f56000     0x7f7067f57000 ---p     1000   2a000 /lib/x86_64-linux-gnu/ld-2.31.so
0x7f7067f57000     0x7f7067f58000 ---p     1000       0 [load19]
0x7ffef802e000     0x7ffef804f000 rw-p    21000       0 [stack]
0x7ffef81ca000     0x7ffef81ce000 r--p     4000       0 [maybe: vvar/vvar_vclock]
0x7ffef81ce000     0x7ffef81d0000 r-xp     2000       0 [vdso]
ffffffffff600000 0xffffffffff601000 r-xp     1000       0 [vsyscall]
pdbg>
```

可以看到 load6 是 heap 段(因为靠近代码段，且大小为 0x21000 这是 libc 初始化分配 heap 的初始值为 0x1000(地位的管理数据)+ 0x20000(实际数据) 组成。

然后还有一个/mnt/flag 段。我们看看这个内存里有什么吧，一般都字符串。

```
pwndbg> x /s 0x7f7067f54000
0x7f7067f54000: "b5xo1ar_gmcrirttdne8ohsoshf6t/:ewm1ea8a_2h4r/ms7cetH6nig2oss}__eyer6aot{_ea_/peot./4-2tho.fb7Bac7_rerr_
_/su1_n!5p8e/c.7Tlcm_saos_sdm_rpa6p3tl2recty_22ffemef_i0e03u-vl5enh9deru"
```

可以看到这里有一段像乱码的字符串，我估计是通过 file 文件里的 flag 值，然后计算出这堆机密后的乱码。估计是让我逆推算法吧。或者是 Z3 求解。上 ida pro 分析看看。

0X10

```
v3 = open(argv[1], 0, envp);
v4 = v3;
if ( v3 == -1 )
{
  perror("Opening file");
}
else
{
  flag = mmap(0LL, 0x1000uLL, 3, 2, v3, 0LL);
  v6 = (__int64)flag;
  if ( flag )
  {
    *(_QWORD *)((char *)flag + 175) = 0LL;
    flag[511] = 0LL;

    memset(
      (void *)(((unsigned __int64)flag + 183) & 0xFFFFFFFFFFFFFFF8LL),
      0,
      8LL * (((unsigned int)flag - (((_DWORD)flag + 183) & 0xFFFFFFF8) -
    inited = (__int64 *)init_shuffle_list((char *)flag);
    *(_QWORD *)v6 = 0LL;
    *(_QWORD *)(v6 + 167) = 0LL;
    memset(
      (void *)((v6 + 8) & 0xFFFFFFFFFFFFFFF8LL),
      0,
      8LL * (((unsigned int)v6 - (((_DWORD)v6 + 8) & 0xFFFFFFF8) + 175)
    shuf(inited, v6);
    puts((const char *)v6);
    kill(0, 11);
    return 0;
  }
```

可以看到确实是从这个 flag 文件里拿到 flag，然后通过 init_shuffle_list 和 shuf 进行加密。这题还挺善良啊，这两个函数名还给我保留了。

0X20

```
v1 = open("/dev/urandom", 0);
v6[0] = 0LL;
v2 = flag;
while ( 1 )
{
  do
    read(v1, &buf, 1uLL);
  while ( buf > 0xAEu );
  v3 = buf;
  if ( !(unsigned __int8)pos_in_list((__int64 *)v6[0], buf) )
  {
    append_list((_QWORD **)v6, v3, *v2++);
    if ( v2 == flag + 175 )
      break;
  }
}
close(v1);
```

根据以上这个图可以知道，是从真实随机数发生器里面拿到随机数，然后看看这个这个随机数如果<= 0XAE 就看看这个随机数在不在 post_list 里面，如果不在的化，就走 append_list 函数。

```
    while ( v4 );
    v6 = malloc(0x10uLL);
    *v5 = v6;
    *v6 = 0LL;
    *(_BYTE *)(*v5 + 8LL) = buf;
    result = (_BYTE *)*v5;
    *(_BYTE *)(*v5 + 9LL) = flag_char;
  }
  else
  {
    result = malloc(0x10uLL);
    *(_QWORD *)result = 0LL;
    result[8] = buf;
    result[9] = flag_char;
    *ss = result;
  }
```

这个 append_list 就是单纯吧随机数放到 v5+8 的地方，把 flag_char 放到 v5+9 的地方，就是把随机数和 flag 存到了 malloc(0x10ull)开辟的结构体中。

0X30

```
__int64 __fastcall shuf(__int64 *flag_ss, __int64 encode_flag)
{
  __int64 result; // rax

  for ( ; flag_ss; flag_ss = (__int64 *)*flag_ss )
  {
    result = *((unsigned __int8 *)flag_ss + 8);
    *(_BYTE *)(encode_flag + result) = *((_BYTE *)flag_ss + 9);
    *((_BYTE *)flag_ss + 9) = 0;
  }
  return result;
}
```

看看这个 shuf 就知道，就是让 encode_flag 中，位置是随机数的位置等于真实的 flag 值，然后再把 flag 在这个内存中的值置 0.

逻辑都已经分析清除了，让我们从 core 中拿到 flag 吧。

因为这个题目是开头是 HTB{…，让我们搜索一下，H 在这个混淆字符串中的哪个位置。

```
└─$ echo $(cat flag) | grep -ob 'H'
51:H
```

然后，还有一件事，因为是用 malloc 不断的顺序小内存，那么它们在内存中的位置很有可能是连续的，让我检查一下堆

```
0x564e4bea31c0: 0x0000000000000000      0x0000000000000000
0x564e4bea31d0: 0x0000000000000000      0x0000000000000000
0x564e4bea31e0: 0x0000000000000000      0x0000000000000000
0x564e4bea31f0: 0x0000000000000000      0x0000000000000000
pwndbg>
0x564e4bea3200: 0x0000000000000000      0x0000000000000000
0x564e4bea3210: 0x0000000000000000      0x0000000000000000
0x564e4bea3220: 0x0000000000000000      0x0000000000000000
0x564e4bea3230: 0x0000000000000000      0x0000000000000000
0x564e4bea3240: 0x0000000000000000      0x0000000000000000
0x564e4bea3250: 0x0000000000000000      0x0000000000000000
0x564e4bea3260: 0x0000000000000000      0x0000000000000000
0x564e4bea3270: 0x0000000000000000      0x0000000000000000
pwndbg>
0x564e4bea3280: 0x0000000000000000      0x0000000000000000
0x564e4bea3290: 0x0000000000000000      0x0000000000000021
0x564e4bea32a0: 0x0000564e4bea32c0      0x0000000000000033
0x564e4bea32b0: 0x0000000000000000      0x0000000000000021
0x564e4bea32c0: 0x0000564e4bea32e0      0x0000000000000077
0x564e4bea32d0: 0x0000000000000000      0x0000000000000021
0x564e4bea32e0: 0x0000564e4bea3300      0x000000000000005d
0x564e4bea32f0: 0x0000000000000000      0x0000000000000021
```

发现了从 0xxxx3290 开始，就有数据了，因为是 malloc 分配的，它肯定是有 16 字节

的管理 header 的，可以看到管理 header 是 0x0 0x21，其中 0x21 代表着，0x20(大小为 32) +0x1(代表前一个邻居不可用或者被占用，如果是 0 代表前一个邻居空闲，之后可以内存整合)

可以看到 0x..32a8 的值是 33，刚好就是 51，那个 H 的值，而且这里有个指针都指向下一块内存，可以看到它指向的内存是连续的。那么我们只需要把它的位置提取出来，然后对 flag 里的值进行重排，就可以拿到正确的 flag 值了。

0X40

用 gdb 的 pi 模式写一个脚本吧。

```
Import os
Flag= “flag”
Start_addr = 0x564e4bea32a8
End_addr = 0x564e4bea4868
Step = 32
With open(file, ‘r’) as f:
    Shuffled_str = f.read().strip()

Inf = gdb.selected_inferior() //获取当前程序状态，并赋值为 inf
For addr in range(start_addr,end_addr+1,step)
    Mem = inf.read_memory(addr,1) //从 addr 的地址读 1 字节的数据
    Index = ord(mem[0])   //因为是一字节，要转为 int，这里发现用这个转 ascii 也可
    Flag += shuffled_addr[index]
Print(flag)
```

```
>>> print(flag)
HTB{by_the_powers_of_https://man7.org/linux/man-pages/man5/core.5.html_i_resurrect_this_process_from_the_dead-reveal_you
r_secrets_to_me!228da2f265e1f3c3c8f4b777600611e822649a}
>>> sS
```

大功告成